

Analisis Fundamentos

Analisis de Trafico

En este apartado se analiza como se generan los datos base del trafico digital. El procedimiento consiste en simular un valor por cada segundo, guardar variables como paquetes, login y salida de datos, y despues revisar si ese comportamiento sigue siendo normal o si fue alterado por un ataque.

Dato por segundo = trafico normal + posible alteracion por ataque

```
for segundo in range(1, 61):  
    paquete = trafico.generar_trafico_normal(segundo)
```

Este bloque es donde empieza la simulacion del trafico. El ciclo recorre 60 segundos y en cada vuelta se genera un paquete normal con informacion del sistema.

```
def generar_trafico_normal(segundo):  
    central = srv.centrales[random.randint(0, 2)]  
    ...  
    paquete = {  
        "segundo": segundo,  
        "paquetes": random.randint(80, 420),  
        "intentos_login": random.randint(0, 2),  
        "cambios_archivos": random.randint(0, 3),  
        "salida_datos": random.randint(1, 12),  
        "incidente": False  
    }  
    return paquete
```

Aqui se crean los datos normales de la red. El paquete guarda datos como solicitudes, intentos de login, cambios de archivos y salida de datos. Como inicia sin ataque, incidente empieza en False.

```
if random.random() < probab:  
    paquete = atk.inyectar_ataque(paquete, segundo)  
    total_incidentes = total_incidentes + 1
```

Este bloque decide si el paquete normal se convierte en ataque. Si el numero aleatorio cae dentro de la probabilidad, se alteran los datos del paquete y se suma un incidente.

```

estado_actual["historial"].append({
    "segundo": segundo,
    "paquetes": paquete["paquetes"],
    "intentos_login": paquete["intentos_login"],
    "salida_datos": paquete["salida_datos"],
    "estado": estado_actual["estado"]
})

```

El historial es donde se guarda lo que paso en cada segundo. Gracias a esto se puede analizar despues el comportamiento de la red, hacer graficas y generar el CSV.

Analisis del comportamiento de la funcion

En este apartado se toma el trafico como una funcion del tiempo. La idea no es encontrar una formula exacta, sino observar como cambia el numero de paquetes conforme pasan los segundos.

$T(t)$ = numero de paquetes en el segundo t

Con esta forma se puede ver si el trafico se mantiene estable, sube de forma gradual o tiene cambios bruscos.

```

for segundo in range(1, 61):
    paquete = trafico.generar_trafico_normal(segundo)

```

Aqui segundo funciona como el tiempo. Cada valor representa un momento diferente de la simulacion, por eso el trafico se puede interpretar como una funcion.

$T(t)$ = numero de paquetes en el segundo t

Esta es la forma matematica de leer los datos. No se busca una formula exacta, sino observar como se comportan los paquetes conforme avanza el tiempo.

```

t.append(h["segundo"])
paquetes.append(h["paquetes"])

```

En las graficas se separa el historial en listas. t guarda el tiempo y paquetes guarda el valor de trafico de cada segundo.

Grafica llamada: 01_paquetes_vs_tiempo.png

Valores o parametros usados:

Valor	Significado
t	tiempo de la simulacion
paquetes	paquetes o solicitudes registradas por segundo
U_PAQUETES_ALERTA	limite para interpretar alerta por paquetes
U_PAQUETES_CRIT	limite para interpretar trafico critico

Explicacion:

Esta grafica representa la funcion principal del trafico. Si la linea sube mucho o cambia de golpe, se puede interpretar como comportamiento irregular o posible ataque.

Grafica llamada: 07_analisis_oscilatorio_vs_tiempo.png

Valores o parametros usados:

Valor	Significado
t	tiempo de la simulacion
paquetes	comportamiento real del trafico
onda_envio	onda regular usada como referencia
intervalo = 5	bloque de tiempo usado para construir la onda

Explicacion:

Esta grafica compara una onda regular contra los paquetes reales. Sirve para ver si el trafico se parece a un envio normal o si se sale demasiado del comportamiento esperado.

Tasa de cambio

En este apartado se calcula cuanto cambia el trafico de un segundo al siguiente. Este procedimiento sirve para detectar subidas repentinas, porque no solo se mira cuantos paquetes hay, sino cuanto cambiaron respecto al dato anterior.

$$\Delta T = T(t) - T(t - 1)$$

Tambien se usa la derivada discreta:

$$\text{Derivada} = (\text{valor actual} - \text{valor anterior}) / \Delta t$$

Como la simulacion avanza de segundo en segundo, Delta t vale 1.

```
metricas_calculo = lm.calcular_metricas_calculo(paquete, estado_calculo)
tasa = metricas_calculo["tasa"]
primera_derivada_sigmoide = metricas_calculo["primera_derivada_sigmoide"]
segunda_derivada_sigmoide = metricas_calculo["segunda_derivada_sigmoide"]
primera_derivada_paquetes = metricas_calculo["primera_derivada_paquetes"]
```

Este bloque toma las metricas de calculo que vienen del archivo matematico. La tasa y las derivadas se separan para poder usarlas en alertas, graficas e historial.

```
def calcular_tasa(actual, anterior):
    return actual - anterior
```

La tasa se calcula restando los paquetes anteriores a los paquetes actuales. Esto representa:

$$\text{Delta } T = T(t) - T(t - 1)$$

Si el resultado es muy alto, significa que el trafico subio de golpe.

```
def calcular_derivada_discreta(actual, anterior, delta_t=1):
    if delta_t == 0:
        return 0
    return round((actual - anterior) / delta_t, 2)
```

Esta funcion calcula el cambio considerando el tiempo. Como el programa avanza de segundo en segundo, delta_t vale 1.

```
if tasa > srv.U_TASA:
    print(" [!!] CRECIMIENTO ABRUPTO DEL TRAFICO  tasa=" + str(tasa))
if segunda_derivada_sigmoide > srv.U_TASA:
    print(" [!!] ACELERACION DEL TRAFICO DETECTADA  d2=" + str(segunda_derivada_sigmoide))
```

Aqui la tasa y la segunda derivada ya se usan como criterio de alerta. Si pasan el umbral, el sistema avisa que hubo un cambio fuerte en el trafico.

Reglas logicas de decision

En este apartado se usan reglas logicas para decidir el estado del sistema. El procedimiento consiste en comparar los datos contra umbrales y combinar condiciones con and y or.

```
p = trafico elevado
q = intentos de login altos
r = flujo critico
```

```
(p AND q) -> ALERTA
(p OR r) -> CRITICO
```

Estas reglas convierten los datos numericos en decisiones del sistema.

```
def evaluar_estado(paquete, flujo, temperatura, cifrado_actual, indice_oscilatorio):
    alertas = []
    protecciones = []
    estado = "NORMAL"
    cifrado = cifrado_actual
    refrigeracion = False
```

Esta funcion es donde el sistema decide si sigue normal o si pasa a sospechoso, alerta o critico. Primero inicia todo en normal y sin nuevas protecciones.

```
if paquete["paquetes"] > srv.U_PAQUETES_ALERTA and paquete["intentos_login"] > srv.U_LOGIN_ALERTA:
    alertas.append("TRAFICO MASIVO + FUERZA BRUTA")
    estado = "ALERTA"
```

Esta regla usa and, o sea que las dos condiciones deben cumplirse. Si hay muchos paquetes y tambien muchos intentos de login, el sistema lo toma como alerta.

```
(p AND q) -> ALERTA
```

Esta es la misma idea escrita en forma logica. p seria trafico elevado y q seria errores o intentos de login excesivos.

```
if paquete["paquetes"] > srv.U_PAQUETES_CRIT or flujo > srv.U_FLUJO_CRITICO:
    estado = "CRITICO"
    cifrado = "REFORZADO"
```

Esta regla usa or, entonces basta con que una condicion sea critica para subir el estado. Si el trafico o el flujo digital se salen demasiado, se activa el modo reforzado.

```
if temperatura >= srv.U_TEMP_CRITICA:
    estado = "CRITICO"
    refrigeracion = True
    protecciones.append("REFRIGERACION ACTIVADA")
```

Esta condicion conecta el analisis con la proteccion fisica. Si la temperatura es critica, el sistema activa refrigeracion.

```
return estado, alertas, protecciones, cifrado, refrigeracion
```

Al final la funcion regresa las decisiones tomadas. El archivo principal usa esos datos para actualizar el sistema.

Adaptacion del simulador

En este apartado se muestra como el simulador se adapto para que no solo genere datos, sino que tambien tome decisiones y produzca resultados. El procedimiento general es: generar paquete, analizarlo, cambiar el estado, activar protecciones y guardar los datos.

Paquete -> metricas -> reglas -> estado -> protecciones -> historial -> graficas y CSV

```
estado_actual["estado"] = est
estado_actual["cifrado"] = cifrado
estado_actual["refrigeracion"] = refrig
```

Aqui el programa actualiza el estado real del sistema con lo que decidieron las reglas logicas.

```
paquete["direccion_registro"] = lm.generar_direccion_registro(
    paquete, estado_actual["estado"], estado_actual["cifrado"], segundo
)
paquete["direccion_servidor"] = lm.descifrar_direccion_registro(
    paquete["direccion_registro"], estado_actual["cifrado"]
)
```

Esta parte adapta el simulador para guardar registros con cifrado y tambien recuperar la ruta original como si la leyera el servidor autorizado.

```

if estado_actual["refrigeracion"]:
    paquete["temperatura"] = lm.aplicar_enfriamiento(
        paquete["temperatura"], v2, lm.P1_REFRIGERANTE, P2, objetivo_termico
    )
else:
    paquete["temperatura"] = lm.aplicar_enfriamiento_pasivo(
        paquete["temperatura"], segundo
    )

```

Aqui se adapta la simulacion para incluir enfriamiento. Si el sistema esta en riesgo, se aplica enfriamiento activo; si no, solo baja de forma pasiva en ciertos intervalos.

Salidas generadas al final de la simulacion:

```

| Salida | Que contiene |
|---|---|
| Graficas PNG | imagenes individuales con paquetes, login, temperatura, estados, flujo, derivadas y oscilacion |
| Reporte CSV | tabla con el historial completo de los 60 segundos |

```

Al terminar la simulacion, el historial se convierte en graficas y en un reporte CSV. Esto permite revisar los datos despues de ejecutar el programa sin volver a correr el monitoreo.

Cifrado Multiplicativo

En este apartado se aplica cifrado para proteger el nombre del registro cuando el sistema ya no esta en estado normal. El procedimiento matematico usa modularidad para que las letras siempre queden dentro del alfabeto y los numeros siempre queden entre 0 y 9.

$$C = aP \bmod 26$$

Donde P es la posicion original de la letra, a es la clave multiplicativa y C es la posicion cifrada.

Para numeros se usa:

$$C = (aP + b) \bmod 10$$

Donde el modulo 10 mantiene el resultado como una cifra del 0 al 9.

```

def cifrado_multiplicativo(texto, clave_letras=5, clave_numeros=7, ajuste_numeros=3):

```

```

resultado = ""
for c in texto:
    if c.isalpha():
        posicion = ord(c) - base
        nueva_posicion = (posicion * clave_letras) % 26
    elif c.isdigit():
        posicion = int(c)
        nueva_posicion = (posicion * clave_numeros + ajuste_numeros) % 10

```

Este bloque aplica el cifrado multiplicativo. En letras usa modulo 26 porque el alfabeto tiene 26 posiciones. En numeros usa modulo 10 porque las cifras van del 0 al 9.

$$C = aP \bmod 26$$

Esta es la formula del cifrado multiplicativo para letras. P es la posicion original, a es la clave y C es la posicion cifrada.

```

def cifrado_reforzado(texto):
    paso1 = cifrado_cesar(texto, 3)
    paso2 = cifrado_multiplicativo(paso1)
    return paso2

```

El cifrado reforzado primero aplica Cesar y despues aplica el multiplicativo. Por eso el dato queda mas alterado cuando el sistema esta en estado critico.

```

if estado == "CRITICO" or cifrado == "REFORZADO":
    nombre_seguro = cifrado_reforzado(nombre_base)
    return "registro_reforzado://" + central + "/" + carpeta + "/" + nombre_seguro

```

Cuando el sistema esta en critico, el registro se guarda con cifrado reforzado. Asi el nombre del archivo ya no queda visible de forma directa.

Interpretacion Matematica

En este apartado se explica como el sistema puede recuperar el dato original. Para descifrar un cifrado multiplicativo se necesita el inverso modular de la clave.

$$a * a^{-1} = 1 \bmod m$$

Si existe ese inverso, se puede regresar de la posicion cifrada a la posicion

original.

$$P = C * a^{-1} \bmod 26$$

En el caso de numeros, primero se quita el ajuste y despues se aplica el inverso:

$$P = (C - b) * a^{-1} \bmod 10$$

```
def obtener_inverso_modular(clave, modulo):  
    for posible in range(1, modulo):  
        if (clave * posible) % modulo == 1:  
            return posible  
    return None
```

Esta funcion busca el inverso modular. El inverso sirve para poder deshacer el cifrado multiplicativo y recuperar el dato original.

```
def descifrado_multiplicativo(texto, clave_letras=5, clave_numeros=7, ajuste_numeros=3):  
    inverso_letras = obtener_inverso_modular(clave_letras, 26)  
    inverso_numeros = obtener_inverso_modular(clave_numeros, 10)
```

Aqui se calculan los inversos que permiten descifrar. Para letras se trabaja modulo 26 y para numeros modulo 10.

```
nueva_posicion = (posicion * inverso_letras) % 26  
nueva_posicion = ((posicion - ajuste_numeros) * inverso_numeros) % 10
```

Estas operaciones recuperan la posicion original. En numeros primero se quita el ajuste y luego se multiplica por el inverso modular.

```
def descifrado_reforzado(texto):  
    paso1 = descifrado_multiplicativo(texto)  
    paso2 = descifrado_cesar(paso1, 3)  
    return paso2
```

El descifrado reforzado hace el proceso al reves. Primero quita el cifrado multiplicativo y despues quita el cifrado Cesar.

```
datos_05.log -> epghb_38.sht -> datos_05.log
```

Este ejemplo muestra que el dato puede cifrarse y despues recuperarse. Esa recuperacion es posible porque las claves usadas tienen inverso modular.

Graficas del analisis

En fundamentos matematicos las graficas sirven para comprobar visualmente si las reglas del sistema tienen sentido. No solo se usan como imagen final, sino como una forma de verificar si los datos, estados y alertas se relacionan correctamente.

Historial -> listas de datos -> graficas -> interpretacion del comportamiento

Grafica llamada: 01_paquetes_vs_tiempo.png

Valores o parametros usados: tiempo, paquetes, umbral de alerta y umbral critico de paquetes.

Explicacion: ayuda a ver si el trafico se mantiene dentro de un comportamiento normal o si tiene subidas fuertes. Para fundamentos, esta grafica se relaciona con la identificacion de datos que pueden pertenecer al conjunto de comportamiento normal o al conjunto de comportamiento sospechoso.

Grafica llamada: 02_intentos_login_vs_tiempo.png

Valores o parametros usados: tiempo, intentos de login, umbral de alerta y umbral critico de login.

Explicacion: permite observar errores o intentos repetidos de acceso. Esta parte apoya las reglas logicas donde los intentos de login altos pueden activar alerta o bloqueo.

Grafica llamada: 04_estado_sistema_vs_tiempo.png

Valores o parametros usados: tiempo y estado convertido a numero: NORMAL = 0, SOSPECHOSO = 1, ALERTA = 2, CRITICO = 3.

Explicacion: muestra como el sistema cambia entre estados. En fundamentos esto se puede explicar como una clasificacion logica: segun las condiciones que se cumplan, el dato cae en una categoria diferente.

Grafica llamada: 05_indice_flujo_digital_vs_tiempo.png

Valores o parametros usados: tiempo, flujo digital, umbral elevado, umbral de

riesgo y umbral critico.

Explicacion: resume varias variables en un solo indicador. Esto ayuda a justificar que las reglas no dependen de un solo dato, sino de la combinacion de diferentes elementos del paquete.

Grafica llamada: 06_derivadas_trafico_vs_tiempo.png

Valores o parametros usados: tiempo, primera derivada con sigmoide, segunda derivada con sigmoide y umbral de tasa.

Explicacion: aunque esta grafica pertenece principalmente a calculo, en fundamentos tambien ayuda a interpretar reglas de decision. Si el cambio del trafico es demasiado alto, el sistema lo puede tomar como una condicion para emitir alerta.

Resultados y analisis

El resultado principal en fundamentos matematicos es la construccion de reglas logicas para clasificar el comportamiento del sistema. A partir de datos como paquetes, login, flujo, temperatura e indice oscilatorio, el programa decide si el estado es normal, sospechoso, alerta o critico.

Las graficas ayudan a revisar si esas decisiones tienen coherencia. Por ejemplo, si los paquetes suben demasiado y el estado tambien sube a alerta o critico, entonces la regla aplicada se puede justificar visualmente. Si una grafica muestra un cambio fuerte pero el sistema no cambia de estado, eso permite detectar una posible limitacion del modelo.

Tambien se observa que el sistema trabaja con conjuntos de datos: usuarios autorizados, usuarios sospechosos, IPs internas, IPs sospechosas e IPs bloqueadas. Esto permite explicar relaciones de pertenencia, clasificacion y reglas de decision.

Conclusiones

Fundamentos matematicos se refleja en el proyecto mediante el uso de proposiciones, reglas logicas, condiciones, conjuntos y clasificacion de estados. El sistema toma datos simulados y los convierte en decisiones usando estructuras como if, and, or, comparaciones y umbrales.

Las graficas complementan el analisis porque permiten ver si las reglas producen resultados coherentes. No solo muestran datos, tambien ayudan a interpretar cuando un comportamiento puede considerarse normal, sospechoso o critico.

La principal limitacion es que las reglas dependen de umbrales definidos por el

proyecto. Si se usaran datos reales de una empresa o de un caso como PEMEX, esos umbrales podrian ajustarse mejor. Aun asi, el prototipo cumple con mostrar como una decision matematica y logica puede convertirse en una alerta dentro de un sistema basico de ciberseguridad.